

Automação SlimFit — Documentação do Sistema (v2 · Nuvem/VPS)

Studio SlimFit — Setor Bueno, Goiânia/GO Integração ZEE ↔ EVO (W12) + Formulário Web + WhatsApp + IA de atendimento (Sofia) Documento atualizado após a migração para servidor online (VPS).

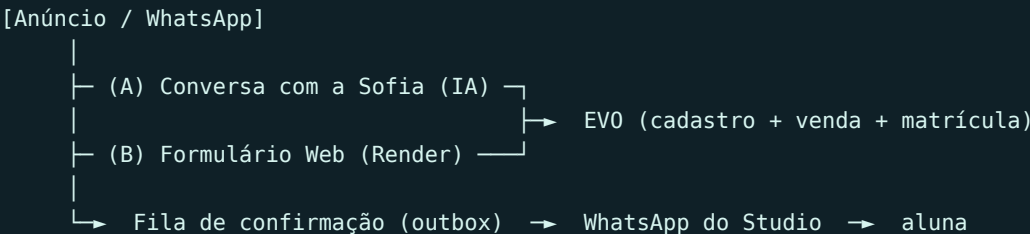
1. Visão geral

O sistema automatiza a jornada da **aula experimental** (do primeiro contato à confirmação) e várias **rotinas de relacionamento e gestão** do Studio, integrando:

- **ZEE** (IA de atendimento no WhatsApp) → descobre quem agendou;
- **EVO / W12** (sistema de gestão) → cadastro, venda, matrícula, relatórios;
- **Formulário Web** (hospedado no Render) → segunda porta de entrada;
- **WhatsApp do Studio** → confirmações, follow-ups, avisos e grupos da equipe;
- **Sofia** → chatbot de IA que atende e agenda pelo WhatsApp;
- **Google Sheets** → planilha de alunas/aniversários.

Mudança principal desta versão: tudo agora roda **online, num servidor VPS (Ubuntu 24.04)**, 24/7, sem depender do PC do Studio ligado. Não há mais Watchdog do Windows nem perfis do Edge.

Arquitetura atual



Servidor VPS (Ubuntu + PM2)

- ├─ Agendador Node (node-cron) → todos os jobs de WhatsApp/EVO
- ├─ Cliente WhatsApp único e persistente (whatsapp-web.js)
- ├─ Job Python ZEE→EVO (cron do sistema, a cada 5 min)
- └─ Ponte com a nuvem (puxa confirmações do formulário a cada 1 min)

2. Componentes do projeto

Pasta	O que é	Tecnologia
Experimental/	O "robô" do Studio: scrapers do EVO, envios de WhatsApp, todos os jobs agendados, ponte com o formulário.	Node.js + Puppeteer + whatsapp-web.js

Pasta	O que é	Tecnologia
Experimental/src/agendamento_evo/	Fluxo automático ZEE → EVO (agenda a experimental no EVO a partir da conversa do ZEE).	Python
ChatBot/	A Sofia — IA que conversa no WhatsApp e agenda a experimental.	Node.js + Anthropic (Claude)
Formulário Web	Segunda porta de entrada (site com calendário e disponibilidade real).	Flask (Python) no Render

3. Infraestrutura (o que roda onde)

Item	Onde / Como
Servidor	VPS Ubuntu 24.04 (ex.: Hostinger), fuso <code>America/Sao_Paulo</code>
Gerenciador de processos	PM2 (mantém o robô no ar, reinicia se cair, sobe no boot)
Navegador	Chromium headless/headed sob xvfb (tela virtual, para o scraper do EVO)
WhatsApp	whatsapp-web.js (sessão salva em disco — escaneia o QR uma vez)
Fluxo ZEE → EVO	cron do sistema , a cada 5 min (<code>rodar-vps.sh</code>)
Formulário	Render (plano grátis), link HTTPS para o anúncio
Sofia (IA)	Node + Anthropic API (modelo Claude Sonnet)
Planilha	Google Sheets via conta de serviço (<code>service-account.json</code>)
Credenciais	Em <code>.env</code> e arquivos locais — nunca versionados

4. Cliente WhatsApp único e persistente (mudança-chave)

Antes, cada job abria/fechava o navegador do WhatsApp — o que derrubava a sessão. Agora há **um único cliente persistente** (`src/wa-client.js`), que sobe **uma vez** junto com o robô e fica autenticado em memória. Todos os jobs **reaproveitam** essa mesma sessão.

- **Sessão salva** (LocalAuth): escaneia o QR **uma vez** e a sessão fica gravada em disco.
- **Evento ready** : os envios só liberam quando o WhatsApp Web terminou de sincronizar.
- **Keep-alive** (a cada ~2h) para manter a conexão "quente".
- **Reinício gracioso** todo dia às **03:00** (fecha e reabre a sessão com segurança).
- **Resiliência a recargas**: se o WhatsApp Web recarregar e "desanexar o frame", os envios **tentam de novo automaticamente** (retry) em vez de falhar.
- **Envio em grupo** por leitura leve da lista (evita erros de serialização) + **@menção nativa**.
- **Resolução de número** (trata o novo "LID" do WhatsApp e valida se o número existe).

5. Fluxo automático ZEE → EVO (Python, a cada 5 min)

1. **Descoberta** — varre as conversas recentes do ZEE, filtra quem tem a tag "FX 3 - Agendou AE" e ainda não "FX 4 - Feito". Idempotente (não reprocessa).
2. **Leitura** — lê nome, e-mail, dia e hora; interpreta linguagem natural ("segunda às 8h15", "amanhã 07:00") para uma data real, sempre daqui pra frente.
3. **Agendamento no EVO** (espelha o processo manual): **Cadastro** → **Venda** do serviço "Aula Experimental" (R\$ 0) → **Matrícula** na turma do horário (limite de 9 alunas/turma).
4. **Exceções** (avisa a recepção): turma lotada, horário inexistente, máx. 2 experimentais/turma, fora da janela, aluna já matriculada.
5. **Fechamento** — marca "FX 4 - Feito", remove "FX 3 - Agendou AE" e **enfileira a confirmação** da aluna para envio pela linha do Studio.

6. Formulário Web (Flask no Render)

- Campos com máscara e validação (Nome, CPF, Telefone, E-mail, Nascimento).
- Calendário dos próximos 10 dias com **disponibilidade real do EVO** (turma com ≤ 7 aparece; 8–9 fica indisponível).
- No envio: revalida a vaga, **bloqueia 2ª experimental** da mesma pessoa (CPF/e-mail/telefone), faz cadastro + venda + matrícula no EVO e **enfileira a confirmação**.
- A chave do EVO fica **só no servidor**.

7. Confirmação para a aluna — WhatsApp do Studio

- Toda marcação (ZEE ou formulário) gera uma **confirmação personalizada** (com endereço e link do Maps).
- Sai pela **linha do Studio**, por uma **fila (outbox)**: grava e o robô envia, com pausa entre mensagens e sem reenviar.
- **Ponte com a nuvem** (a cada **1 min**): puxa as confirmações do formulário (Render) para a fila local. O intervalo curto evita a perda quando o Render (grátis) reinicia e mantém a nuvem "acordada". (*Substitui a antiga ponte em Python do PC.*)

8. Agenda de envios (jobs do robô)

Todos rodam no fuso de São Paulo, dentro do agendador (PM2 + node-cron).

Job	Horário	Dias	O que faz / destino
Ponte formulário (nuvem)	a cada 1 min	Todos	Puxa confirmações do formulário → fila local
Confirmação Experimental	a cada 15 min	Todos	Envia a fila (ZEE + formulário) pela linha do Studio
Reinício gracioso do WhatsApp	03:00	Todos	Renova a sessão do WhatsApp com segurança
Aniversariantes do mês	05:30	Dia 28	Lista do mês seguinte no grupo da equipe
Ausentes 10 dias	06:10	Segunda	Alunas ativas sem presença há 10+ dias → grupo da equipe
Presentes pendentes	06:45	Segunda	Presentes de tempo de casa pendentes → grupo
Instagram boas-vindas	07:00	Todos	DM a novos seguidores (<i>desligado no VPS — ver §11</i>)
Aniversariantes do dia	08:00	Todos	Parabéns nos grupos em comum, com @menção

Job	Horário	Dias	O que faz / destino
Confirmação — HOJE	08:30	Seg-Sáb	Experimentais de hoje (status Agendado)
Follow-up manhã	10:30	Seg-Sáb	Texto + áudio da professora (aula de ontem, manhã)
Faltas (no-show) manhã	11:30	Seg-Sáb	Quem faltou de manhã → convite para remarcar
Planilha de aniversários	14:00	Todos	Atualiza o Google Sheets (não envia msg)
Confirmação — AMANHÃ	15:30	Dom-Sex	Experimentais de amanhã (status Agendado)
Follow-up tarde	16:00	Seg-Sáb	Texto + áudio da professora (aula de ontem, tarde)
Resumo da semana	16:30	Sexta	Funil das experimentais no grupo
Renovação de contratos	17:00	Segunda	Contratos vencendo na semana (perfil Financeiro)
Faltas (no-show) tarde/noite	19:30	Seg-Sáb	Quem faltou à tarde/noite → convite para remarcar
Resumo do dia	19:45	Todos	Resumo completo do dia no grupo da equipe (ver §9)

Detalhes de alguns jobs

- **Follow-up (com áudio):** no dia seguinte, para quem teve **Presença**. Escolhe o áudio da professora pela escala; envia como **mensagem de voz** (áudios em OGG/Opus). Mensagem diferente para quem já virou aluna vs. quem ainda não fechou.
- **Aniversariantes do dia:** usa a API nativa de "grupos em comum" e marca a aniversariante com **@menção** real.
- **Ausentes 10 dias:** lê o CRM > Faltantes do EVO ("sem presença nos últimos 10 dias" + "Contabilizar Inadimplentes"), confere aluna por aluna se o **contrato está ATIVO**, e manda a lista no grupo.
- **Renovação:** só contratos com vencimento do contrato **atual** na semana vigente.

9. Resumo do dia (novo — 19:45)

Mensagem diária para o grupo da equipe, com 5 blocos:

1. 🔄 **Reposições** — nome, horário e **status** (Presença / Falta / **Falta Justificada**), lidos da tela **Grade > Horários** (visão do dia).
2. ✅ **Experimentais que fizeram aula** — nome + horário.
3. ❌ **Experimentais que faltaram** — nome + horário.
4. 🎉 **Fechou contrato** — **todas** as alunas que fecharam contrato no dia (com ou sem experimental), lidas de **Gerencial > Vendas** (nome + contrato).
5. ✂️ **Rescisões** — cancelamentos do dia, lidos de **Gerencial > Cancelamentos** (nome + contrato).

Se o dia não tiver nada relevante, não envia (evita resumo vazio). Tem **retry** contra a instabilidade do login do EVO.

10. Sofia — chatbot de IA (WhatsApp)

- Atende no WhatsApp, entende a conversa e **agenda a experimental** no EVO.
- Modelo **Claude (Sonnet)** via Anthropic API; mantém o contexto da conversa (sessão por contato).
- **Prompt controlado por um editor** (arquivo de prompt + mídias): tom curto e humano, preço correto, saudação obrigatória na 1ª resposta, e **guarda-corpos anti-invenção** (só afirma o que está escrito).
- Imagens (grade/preços) e textos configuráveis sem mexer no código.

11. Instagram — status atual

O job de boas-vindas por DM **existe e está pronto**, mas **desligado por padrão no VPS** (`IG_ENABLED=false`).

Motivo: a Meta **bloqueia o navegador automatizado a partir de IP de datacenter** (erro HTTP 429), mesmo com o IP "limpo". Caminhos para reativar:

- **IP residencial** (rodar essa rotina num PC de casa), ou
- **Proxy residencial/móvel** (suporte já embutido via `IG_PROXY` no `.env`).

Enquanto isso, as boas-vindas do Instagram podem ser feitas manualmente. O resto do sistema não é afetado.

12. Confiabilidade e segurança

- **PM2** mantém o robô no ar, reinicia sozinho e volta no boot do servidor (`pm2 save` / `pm2 startup`).
 - **Um job por vez** (trava de execução) + **heartbeat** no log.
 - **Reinício gracioso** do WhatsApp às 03:00 + **retry** em erros transitórios de frame.
 - **Idempotência** em todas as etapas (não duplica cadastro, não reprocessa, não reenvia).
 - **Credenciais** em `.env` e `service-account.json` — fora do versionamento. Perfis e dados de sessão também não são versionados.
 - **Editar um arquivo no servidor só passa a valer após reiniciar o processo** (`pm2 restart`), pois o Node mantém os módulos em cache.
-